

Comparação entre Min-Max com Poda Alfa-Beta e Monte Carlo Tree Search no Othello

Trabalho 2 – Inteligência Artificial

Ana Lilian Alfonso Toledo¹, Tobias Viero de Oliveira¹

¹Universidade Federal de Santa Maria

1. Introdução

O jogo *Othello*, também conhecido como *Reversi*, é um cenário canônico para o estudo de busca adversarial em Inteligência Artificial. Caracteriza-se por regras simples mas alta complexidade combinatória, presença forte de inversões de material e fortes assimetrias posicionais (cantos, bordas, casas-X), o que produz um espaço de decisão rico mesmo em tabuleiros reduzidos. Esses elementos tornam o jogo um banco de testes natural para comparar abordagens fundamentalmente diferentes de busca.

Este trabalho desenvolve dois agentes para o Othello e os compara experimentalmente. O primeiro é o Min-Max com poda Alfa-Beta, que encarna a tradição de busca exaustiva guiada por uma função de avaliação. O segundo é o Monte Carlo Tree Search (MCTS), que substitui a heurística estática por estatísticas obtidas via simulação aleatória de partidas até o estado terminal. As duas abordagens partem de premissas distintas sobre o que constitui “conhecer” o valor de uma posição: enquanto o Min-Max demanda uma definição explícita do que é uma boa configuração, o MCTS infere essa definição a partir de muitas amostras.

O objetivo central é comparar essas abordagens em três dimensões: taxa de vitória em confrontos diretos, custo computacional (tempo por jogada, nós expandidos, podas, simulações executadas) e qualidade das decisões medida pela margem de pontuação ao final. Como extensão obrigatória, foi implementado um modo de pontuação ponderada por posição e suporte a tabuleiros de tamanho configurável (4×4 , 6×6 e 8×8), o que permite investigar como a relação entre os dois algoritmos varia com a escala do problema.

O restante do artigo está organizado como segue. A Seção 2 revisa os fundamentos teóricos. A Seção 3 descreve a implementação e o protocolo experimental. A Seção 4 apresenta os resultados. A Seção 5 discute os achados em termos das diferenças conceituais entre busca heurística e busca por simulação. A Seção 6 conclui o trabalho.

2. Fundamentação Teórica

2.1. Othello/Reversi

O Othello é um jogo de soma zero, perfeita informação e turnos alternados [1]. Em cada turno o jogador atual coloca uma peça de sua cor em uma casa vazia tal que essa jogada *flanqueie* ao menos uma sequência contínua de peças adversárias entre a peça recém-colocada e outra peça do próprio jogador, ao longo de uma das oito direções (horizontal, vertical e diagonais). Toda peça flanqueada é virada para a cor do jogador atual. Quando um jogador não possui jogadas legais, ele é forçado a passar; se nenhum dos dois jogadores tem jogadas legais, a partida termina. O vencedor é o jogador com mais peças (ou, no nosso modo *weighted*, o jogador com maior pontuação posicional ponderada).

2.2. Min-Max com poda Alfa-Beta

O algoritmo Min-Max é a formalização clássica de busca em árvores de jogos de soma zero. O jogador MAX escolhe a ação que maximiza sua utilidade assumindo que MIN responderá minimizando-a. Para árvores de profundidade arbitrária a recursão é inviável, e a busca é tipicamente truncada em uma profundidade fixa d , substituindo a utilidade dos estados não-terminais pela saída de uma função de avaliação heurística $h(s)$ [1].

A poda Alfa-Beta [2] preserva a equivalência ao Min-Max sem busca exaustiva, mantendo dois limites α (melhor garantia atual de MAX) e β (melhor garantia atual de MIN). Ramos para os quais $\alpha \geq \beta$ podem ser descartados sem afetar a decisão final na raiz. No melhor caso (jogadas perfeitamente ordenadas), a poda reduz o tempo de busca de $\mathcal{O}(b^d)$ para $\mathcal{O}(b^{d/2})$, o que efetivamente dobra a profundidade alcançável com o mesmo orçamento.

2.3. Monte Carlo Tree Search (MCTS)

O MCTS substitui a função de avaliação estática por estatísticas derivadas de simulações aleatórias [3, 5]. Constrói-se uma árvore incremental ao longo de muitas iterações, cada uma composta por quatro fases:

- **Seleção:** a partir da raiz, navega-se recursivamente escolhendo em cada nó o filho que maximiza a função de seleção, tipicamente UCT [4]: $UCT(c) = Q(c) + C \cdot \sqrt{\ln N(p)/N(c)}$, onde $Q(c)$ é o valor médio observado no filho c , $N(p)$ e $N(c)$ são as visitas do pai e do filho, e C é uma constante de exploração;
- **Expansão:** ao alcançar um nó cujas jogadas ainda não foram todas expandidas, cria-se um novo filho;
- **Simulação (rollout):** a partir do nó recém-expandido, simula-se uma partida até o estado terminal usando uma política leve de escolha de jogadas;
- **Retropropagação:** o resultado terminal (vitória, derrota ou empate) atualiza visitas e médias de todos os nós no caminho percorrido.

A jogada escolhida na raiz após o orçamento de iterações esgotar-se é, em geral, a mais visitada. Diferentemente do Min-Max, o MCTS é *anytime* (pode ser interrompido a qualquer momento e produzir uma decisão) e não exige uma função de avaliação explícita.

3. Metodologia

3.1. Arquitetura da implementação

O projeto foi implementado em Python 3 e organizado em camadas com fronteiras explícitas:

- **game:** motor do Othello (representação do estado, geração de jogadas legais com captura em oito direções, regra de *pass*, condição de término, cálculo de vencedor);
- **evaluation:** heurísticas de estado e políticas de rollout reutilizáveis;
- **agents:** agentes Min-Max e MCTS, sem conhecimento das regras internas do tabuleiro;
- **experiments:** orquestração de partidas, coleta de métricas e exportação de resultados.

Esse desacoplamento garante que extensões de regras (modo *weighted*) e de tamanho de tabuleiro afetem todos os agentes uniformemente sem duplicação de lógica.

3.2. Agente Min-Max com poda Alfa-Beta

A implementação utiliza Min-Max com poda Alfa-Beta e profundidade limitada configurável (no experimento, $d \in \{3, 4, 5\}$). Adicionalmente, foi implementada *ordenação de jogadas* no nível raiz: as jogadas são pré-ordenadas pela heurística de avaliação aplicada ao estado sucessor, o que aumenta a probabilidade de cortes Alfa-Beta precoces.

Por jogada, o agente coleta as seguintes métricas: número de nós expandidos, número de eventos de poda Alfa-Beta, profundidade efetivamente atingida e tempo decorrido. Essas métricas formam a base da análise de custo computacional apresentada na Seção 4.

Trace ilustrativo da poda. Para fins didáticos, o Apêndice A apresenta uma execução completa do algoritmo com profundidade limite 2 partindo do estado inicial padrão do Othello 6×6 , incluindo a árvore de decisão construída (Figura 11) e os valores de α e β propagados em cada nó. Na execução, foram expandidos 10 nós e ocorreram 3 eventos de poda, descartando 6 ramos. A jogada selecionada na raiz foi (1, 2) com valor Min-Max 0, 0.

3.3. Função de avaliação

A função de avaliação tem o formato canônico de uma combinação linear ponderada:

$$\text{Avaliar}(s) = \sum_i w_i \cdot f_i(s), \quad (1)$$

onde cada f_i é uma característica normalizada no intervalo $[-1, 1]$ a partir da perspectiva do jogador de referência. Foram implementadas cinco características:

- **Diferença de peças:** $(p - o)/(p + o)$, mede vantagem de material;
- **Mobilidade:** análoga à diferença de peças, mas com o número de jogadas legais; capta pressão tática;
- **Controle de cantos:** proporção dos cantos do tabuleiro ocupados pelo jogador, descontada a do adversário; cantos são peças permanentemente estáveis no Othello;
- **Controle de bordas:** análoga aos cantos, restrita às casas das bordas (excluindo cantos);
- **Proximidade de canto:** penaliza peças adjacentes a cantos vazios, refletindo a heurística clássica de evitar oferecer cantos ao adversário.

Foram definidas duas composições principais para uso no Min-Max, sintetizadas na Tabela 1.

A escolha dos pesos reflete duas hipóteses estratégicas distintas. A heurística *balanced* distribui peso de modo aproximadamente uniforme entre material (peças), pressão (mobilidade) e estrutura posicional (cantos, bordas), e serve como linha de base. A heurística *aggressive* desloca peso explícito do material imediato para mobilidade e cantos, sob a hipótese de que perder peças temporariamente em troca de pressão e estrutura é vantajoso a médio prazo. Adicionalmente, há uma terceira composição *lightweight* (peças 65%, mobilidade 35%) usada apenas dentro dos rollouts heurísticos do MCTS, para preservar baixo custo de avaliação.

Tabela 1. Pesos das duas composições heurísticas avaliadas. Os pesos foram calibrados para somar 1 e refletir as prioridades estratégicas de cada perfil.

Característica	balanced	aggressive
Diferença de peças	0,30	0,10
Mobilidade	0,25	0,40
Controle de cantos	0,30	0,35
Controle de bordas	0,10	0,05
Proximidade de canto	0,05	0,10

3.4. Agente Monte Carlo Tree Search

O agente MCTS implementa o ciclo completo das quatro fases descritas na Seção 2, com seleção UCT e constante de exploração $C = \sqrt{2} \approx 1,414$. O orçamento de simulações por jogada é configurável (no experimento: 100, 500 e 1000).

Foram implementadas quatro políticas de rollout, em ordem crescente de sofisticação:

- **random**: jogadas escolhidas uniformemente entre as legais, é a política canônica do MCTS clássico;
- **rollout_simple**: prioriza, nessa ordem, ocupação de cantos, ganho de mobilidade e ocupação de bordas; é uma regra leve e baratíssima de calcular;
- **rollout_successor_eval**: aplica uma vez cada jogada legal, avalia o estado sucessor com a heurística *lightweight* e escolhe gulosamente a de maior valor;
- **rollout_topk**: ordena as jogadas pela heurística e amostra uniformemente entre as k melhores, preservando alguma diversidade.

Por jogada, o agente coleta: número de simulações executadas, nós criados na árvore e tempo decorrido.

3.5. Extensão obrigatória

O trabalho implementa duas extensões não-triviais. A primeira é o tamanho de tabuleiro configurável (4×4 , 6×6 e 8×8). Toda a lógica de regras, geração de jogadas, identificação de cantos/bordas e cálculo de heurísticas é dinâmica em relação ao tamanho do tabuleiro, sem hardcode de dimensões. A segunda é o modo de pontuação ponderada (*weighted*), que altera o objetivo estratégico do jogo: em vez de contar peças, soma-se uma pontuação posicional, com cantos valendo 4 pontos, bordas valendo 2 e casas internas valendo 1. Essa modificação afeta diretamente os estados terminais e, portanto, a propagação de utilidade tanto no Min-Max quanto no MCTS.

3.6. Protocolo experimental

Foram executadas 264 partidas distribuídas em 18 cenários distintos, envolvendo 12 configurações de agente. O experimento seguiu três princípios metodológicos: (i) cada confronto é repetido com sementes distintas e cores alternadas para reduzir viés de BLACK/WHITE; (ii) cada par de configurações é testado em isolamento, em contraste

com torneios all-vs-all que misturariam efeitos de matchup; (iii) todos os resultados, incluindo as configurações dos agentes, são serializados em CSV para reprodução e auditoria.

A análise utiliza três métricas principais: *taxa de vitória* (fração de partidas vencidas), *margem normalizada* (definida como $(p - o)/(p + o)$ ao final da partida), e *custo* (tempo médio por jogada, nós expandidos por partida e número de podas para o Min-Max; simulações executadas para o MCTS).

4. Resultados Experimentais

4.1. Visão geral dos confrontos

A Figura 1 sintetiza a taxa de vitória de todos os confrontos diretos restritos ao tabuleiro 6×6 . O agente da linha enfrentou o agente da coluna, e a cor indica a fração de partidas vencidas. Várias relações aparecem com clareza: o Min-Max $d=4$ com heurística *aggressive* venceu o Min-Max $d=4$ *balanced* em 100% dos 16 confrontos; o MCTS escalou monotonicamente com o orçamento (MCTS500 vence MCTS100 em 88% dos jogos, e MCTS1000 vence MCTS500 em 81%); e a política *rollout_simple* venceu *random* em 94% dos jogos.

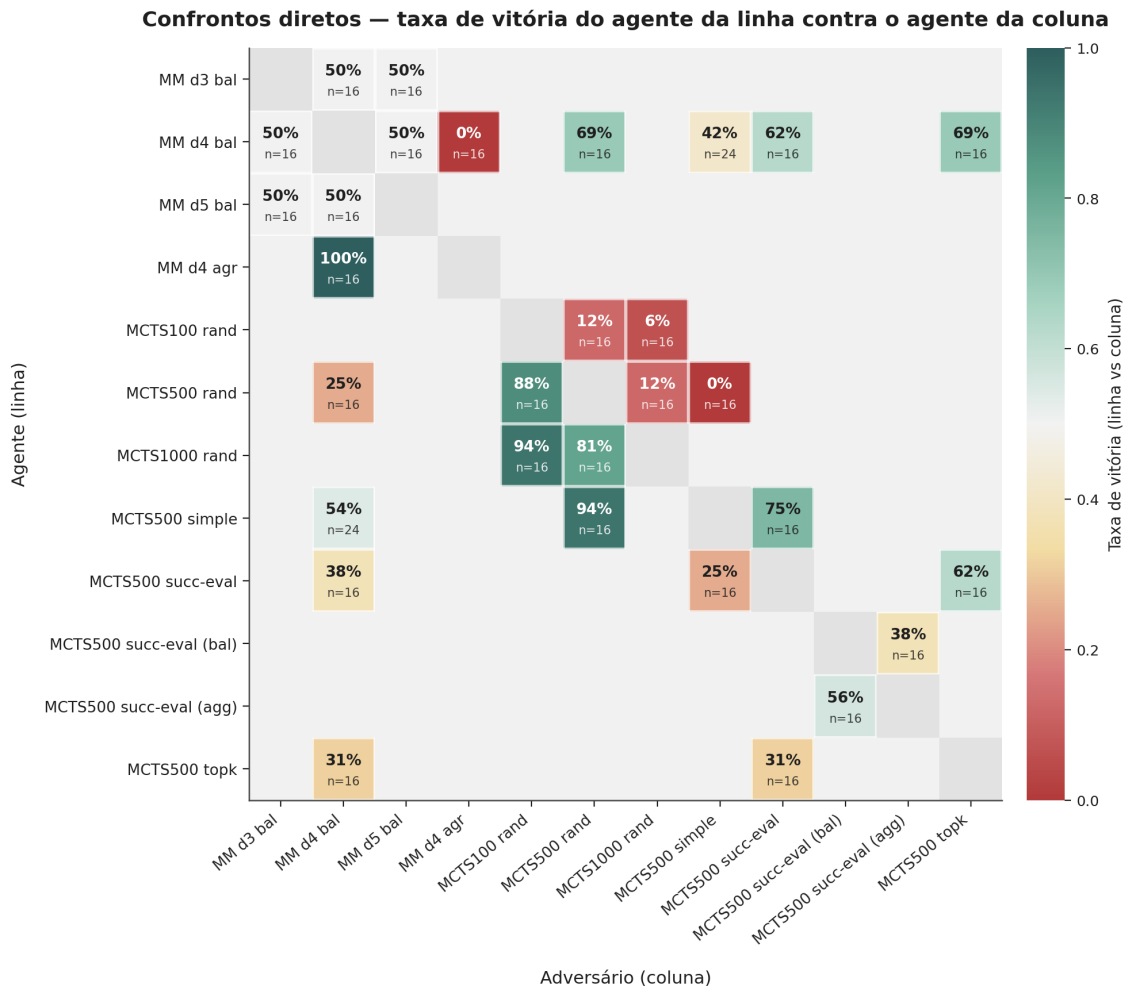
A diagonal entre os Min-Max balanceados de profundidades distintas exhibe um fenômeno surpreendente que será analisado em detalhe a seguir: todos os três confrontos de profundidade pura ($d3 \times d4$, $d4 \times d5$, $d3 \times d5$) fecharam exatamente em 50%/50%.

4.2. Min-Max: impacto da profundidade

A Figura 2 aprofunda os confrontos entre profundidades. O painel (a) mostra que, com a mesma heurística *balanced*, nenhum confronto de profundidade pura produziu vantagem em taxa de vitória — todos terminaram em 8/8. Já o painel (b) revela diferenças escondidas pelo placar: no confronto $d5 \times d4$, a margem normalizada do agente mais profundo é deslocada para o positivo (mediana $\approx +0,20$), sugerindo que o $d5$ vence com folga maior do que perde; no $d5 \times d3$, contudo, a mediana fica em $-0,33$, indicando que apesar do empate no placar, o agente mais profundo perde mais partidas do que vence em magnitude. O painel (c) mostra que o custo cresce de forma quase exponencial: 763 nós em $d=3$, 2.200 em $d=4$ e 6.367 em $d=5$, com tempo médio por jogada de 0,04 s, 0,11 s e 0,32 s respectivamente.

A leitura combinada do painel (b) é importante. Embora a Figura 1 mostre 50% em todos os confrontos de profundidade, o painel de margens deixa explícito que esses 50% encobrem comportamentos distintos: às vezes derrotas folgadas balanceadas por vitórias folgadas ($d5 \times d3$), às vezes uma leve tendência positiva ($d5 \times d4$). Isso sustenta a interpretação de que, no 6×6 com heurística uniforme entre os contendores, a profundidade extra não está convertendo-se em vantagem consistente: o ganho marginal por nível adicional é dominado pelas diferenças induzidas por sementes e cores.

A Figura 3 examina explicitamente o efeito da poda. O painel (a) compara nós expandidos com eventos de poda; o (b) reporta a razão podas/nós, e o (c) a variabilidade entre partidas. A taxa de poda é não-monotônica: 0,19 em $d=3$, 0,29 em $d=4$ e cai de volta para 0,18 em $d=5$. Essa queda em $d=5$ é coerente com o esperado: a ordenação de jogadas no nível raiz mantém-se eficaz nas profundidades intermediárias, mas perde força quando a árvore fica muito profunda, dado que a heurística é estimada apenas no topo.



Confrontos restritos ao tabuleiro 6×6. Células cinzas claras: confronto não testado. Cinza médio: mesmo agente.

Figura 1. Heatmap de confrontos diretos no tabuleiro 6×6. Cada célula mostra a taxa de vitória do agente da linha contra o agente da coluna, com n indicando o número de partidas. Células cinzas correspondem a confrontos não testados. A diagonal principal é vazia por construção.

4.3. MCTS: impacto do número de simulações

A Figura 4 apresenta o efeito do orçamento de simulações. Diferentemente da profundidade no Min-Max, o impacto do orçamento no MCTS é *monotônico* e claro: 500 vence 100 em 88% dos casos, 1000 vence 500 em 81%, e 1000 vence 100 em 94%. As margens (painel b) confirmam o padrão. Há também sinal claro de **retornos decrescentes**: a mediana da margem no confronto 500×100 (+0,45) é maior que no 1000×500 (+0,35), embora o passo de orçamento seja igual (5×). O custo, por sua vez, cresce praticamente linearmente: o tempo médio por jogada vai de 0,54 s a 2,24 s e a 5,83 s, um fator de aproximadamente 10,8× entre os extremos.

4.4. MCTS: efeito da política de rollout

A Figura 5 compara as quatro políticas de rollout. O painel (a) mostra confrontos diretos entre rollouts no mesmo orçamento (500 simulações): *simple* venceu *random* em 94%

Min-Max — Impacto da profundidade nas decisões e no custo computacional

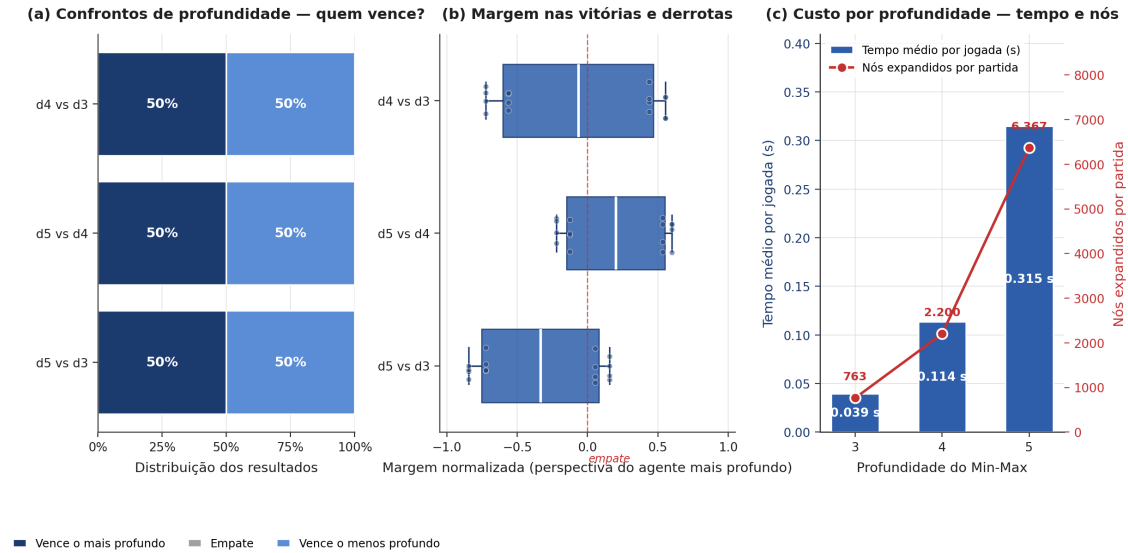


Figura 2. Min-Max e profundidade. (a) Distribuição de resultados nos três confrontos de profundidade pura. (b) Distribuição da margem normalizada medida da perspectiva do agente mais profundo — cada ponto é uma partida. (c) Tempo médio por jogada (barras azuis) e nós expandidos por partida (linha vermelha) por profundidade.

Min-Max — Custo computacional e impacto da poda Alfa-Beta

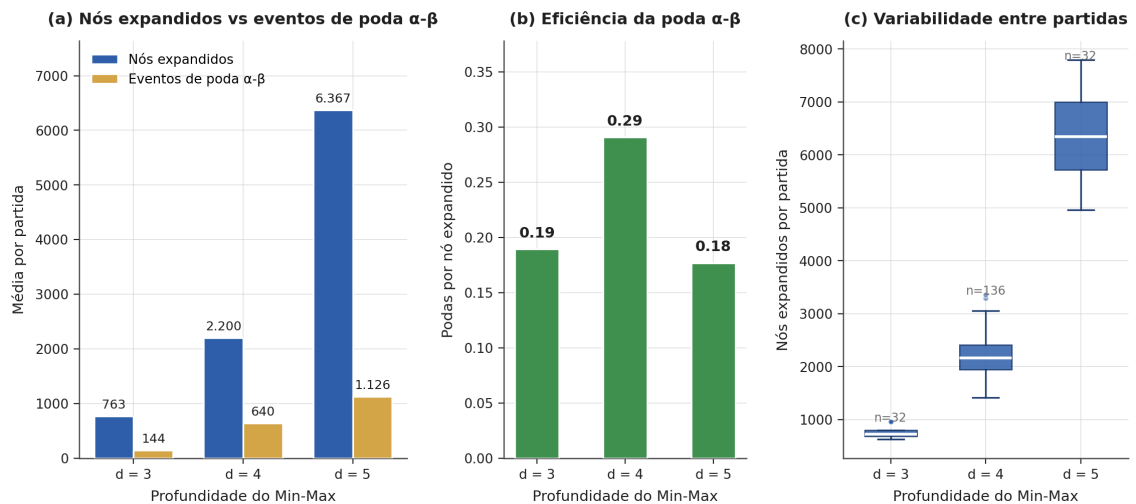


Figura 3. Custo computacional do Min-Max e impacto da poda Alfa-Beta. (a) Nós expandidos versus eventos de poda. (b) Eficiência da poda medida em podas/nó. (c) Boxplots da distribuição de nós expandidos por partida.

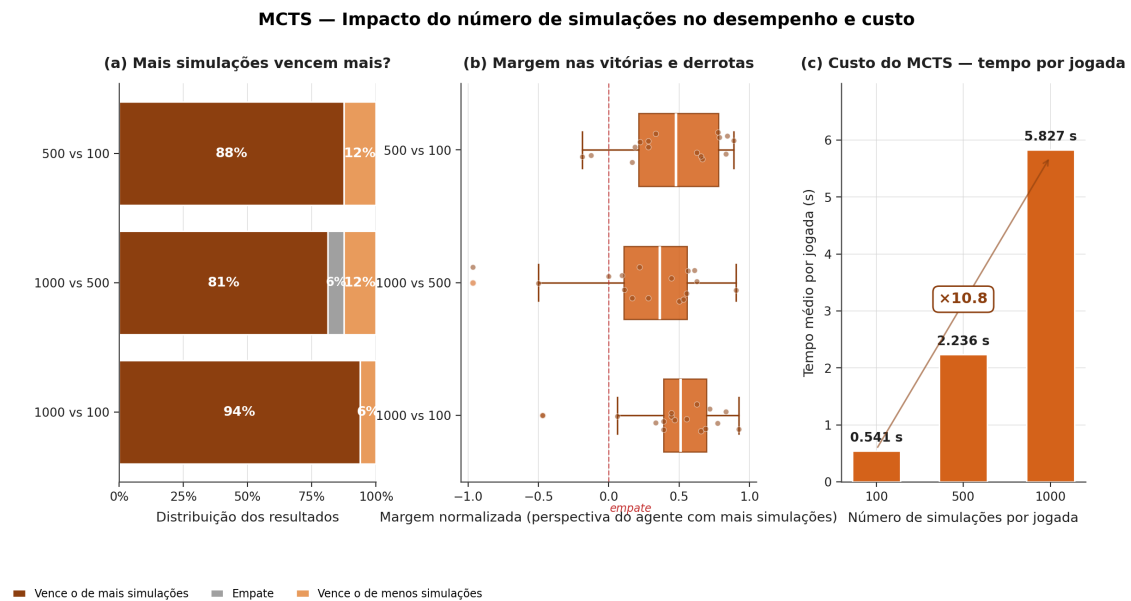


Figura 4. MCTS e número de simulações. (a) Distribuição de resultados. (b) Margem normalizada da perspectiva do agente com mais simulações. (c) Tempo médio por jogada por orçamento.

dos jogos, mas venceu *succ-eval* apenas em 75% e *succ-eval* venceu *topk* em 62%. O painel (b), que avalia cada política contra o mesmo adversário (Min-Max d=4 *balanced*), produz um resultado contraintuitivo: *rollout_simple* é a única política que empata com o Min-Max (50%/50%), enquanto *succ-eval* e *topk*, mais sofisticados, vencem apenas 38% e 31% dos jogos respectivamente.

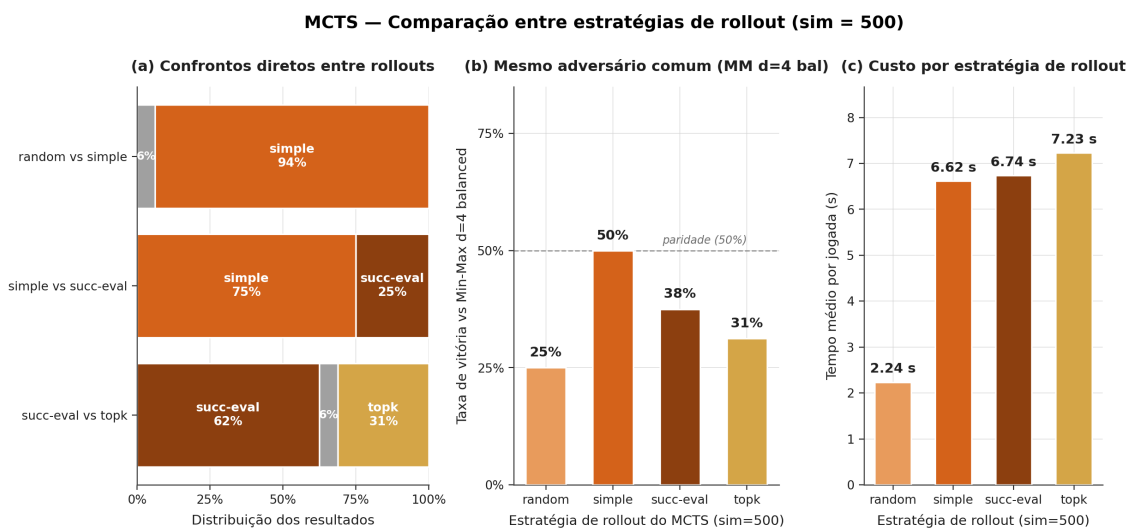


Figura 5. MCTS e estratégia de rollout (sim=500). (a) Confrontos diretos entre rollouts. (b) Cada rollout enfrentando o mesmo adversário comum. (c) Custo médio por jogada por estratégia.

A interpretação razoável desse padrão é que rollouts mais determinísticos (*succ-eval* é guloso) reduzem a diversidade dos playouts e enviesam as estatísticas de UCT.

rollout_simple oferece um meio-termo: usa sinal posicional (cantos, mobilidade, bordas) sem amarrar todas as escolhas ao maior valor heurístico imediato. O painel (c) confirma que essa qualidade não vem de graça em *succ-eval* e *topk*: ambos custam cerca de 6,7 s e 7,2 s por jogada, respectivamente, contra 6,6 s para *simple* e apenas 2,2 s para *random*.

4.5. Heurística *balanced* versus *aggressive*

O resultado mais marcante do experimento aparece na Figura 6. A heurística *aggressive* venceu *balanced* em 100% dos 16 confrontos do Min-Max $d=4$, com mediana de margem normalizada de $+0,65$ (painel b). Para o MCTS na mesma comparação, a vantagem é muito mais discreta: 56% de vitórias, com mediana próxima de zero ($+0,06$). O painel (c) sintetiza: enquanto a média da margem agressiva sobre balanceada é $+0,66$ no Min-Max, ela cai para $-0,03$ no MCTS.

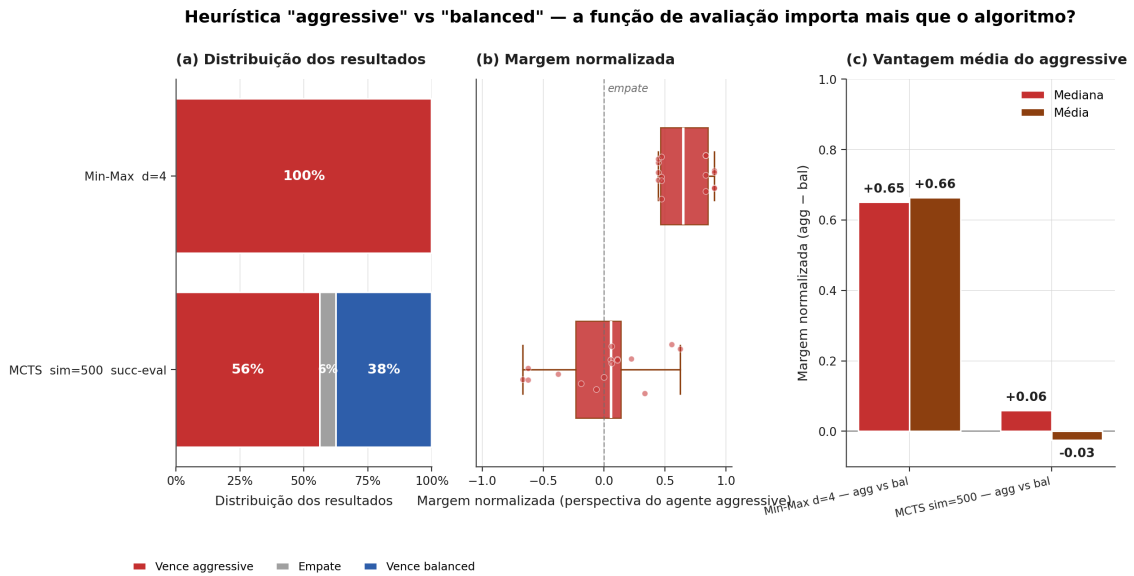


Figura 6. Comparação direta entre as heurísticas *aggressive* e *balanced* para os dois algoritmos. A heurística domina o comportamento do Min-Max mas mal afeta o MCTS.

Esse contraste é, conceitualmente, o achado central do trabalho. No Min-Max, a heurística é a *única* fonte de valor para todos os estados não terminais visitados na busca, e por isso a mudança de pesos altera drasticamente todas as decisões. No MCTS, a heurística participa apenas como guia da política de rollout: o valor de uma posição é majoritariamente estimado pelo resultado das simulações que descem até o estado terminal, no qual a heurística é irrelevante. O MCTS é, portanto, mais robusto a uma heurística mal calibrada, mas também ganha menos com uma heurística boa.

4.6. Custo-benefício e mapa de agentes

A Figura 7 consolida todos os agentes em um mapa custo $\times$ benefício. No eixo x (logarítmico) está o tempo médio por jogada, e no y a taxa de vitória global do agente em todas as suas partidas no 6×6 . O Min-Max $d=4$ *aggressive* ocupa o canto superior esquerdo (rápido e forte) com 100% de vitórias a 0,17 s por jogada. O MCTS1000 *random* (88% a 5,83 s) e o MCTS500 *simple* (71% a 6,62 s) também estão na metade superior,

mas a um custo computacional 30 a 40 vezes maior. As variantes *succ-eval* e *topk* ocupam o quadrante “lento e fraco”, sugerindo que o investimento heurístico no rollout não compensa quando o orçamento de simulação está limitado a 500.

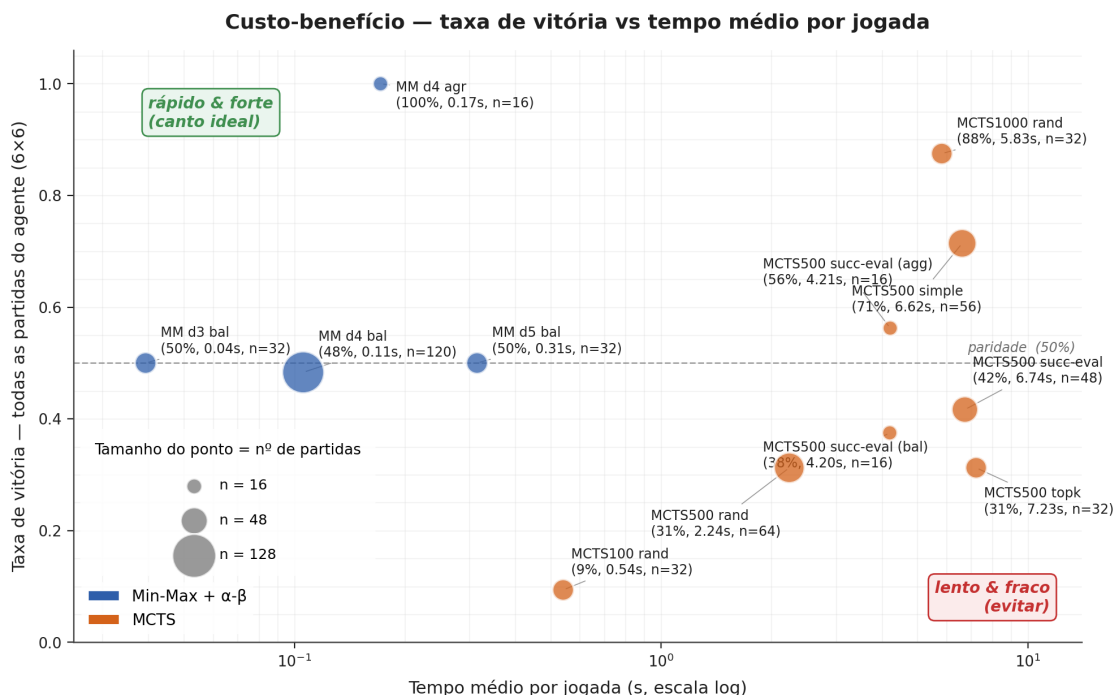


Figura 7. Mapa de custo-benefício. Cada ponto é uma configuração de agente; tamanho do ponto = número de partidas. As taxas de vitória globais devem ser interpretadas com cautela, pois cada agente enfrentou um conjunto diferente de adversários.

4.7. Validade metodológica: viés de cor

A Figura 8 aborda um requisito metodológico do experimento: verificar se a alternância de cores eliminou o viés. Globalmente (painel a), das 264 partidas, BLACK venceu 137 (51,9%), WHITE venceu 121 (45,8%) e ocorreram 6 empates (2,3%). Esse desvio modesto é compatível com a literatura clássica do Othello, em que BLACK tem leve vantagem por jogar primeiro.

O painel (b) revela uma sutileza importante: dois agentes (Min-Max d=3 *balanced* e Min-Max d=5 *balanced*) têm diferença BLACK–WHITE de exatamente +1,00 — vencem 100% como BLACK e 0% como WHITE. Esse resultado *não* é um defeito experimental, mas uma propriedade esperada de busca determinística: quando dois Min-Max determinísticos com a mesma heurística enfrentam-se com a mesma semente de jogo, o resultado é fixo, e a alternância de cor apenas troca quem “ganha” segundo as regras de desempate. Esse efeito não aparece em confrontos envolvendo MCTS porque suas simulações são estocásticas.

4.8. Tamanho do tabuleiro

A Figura 9 avalia como o mesmo confronto (Min-Max d=4 *balanced* versus MCTS sim=500 *rollout_simple*) varia com o tamanho do tabuleiro. No 4×4, com partidas curtas

Viés de cor — o desenho experimental é justo?

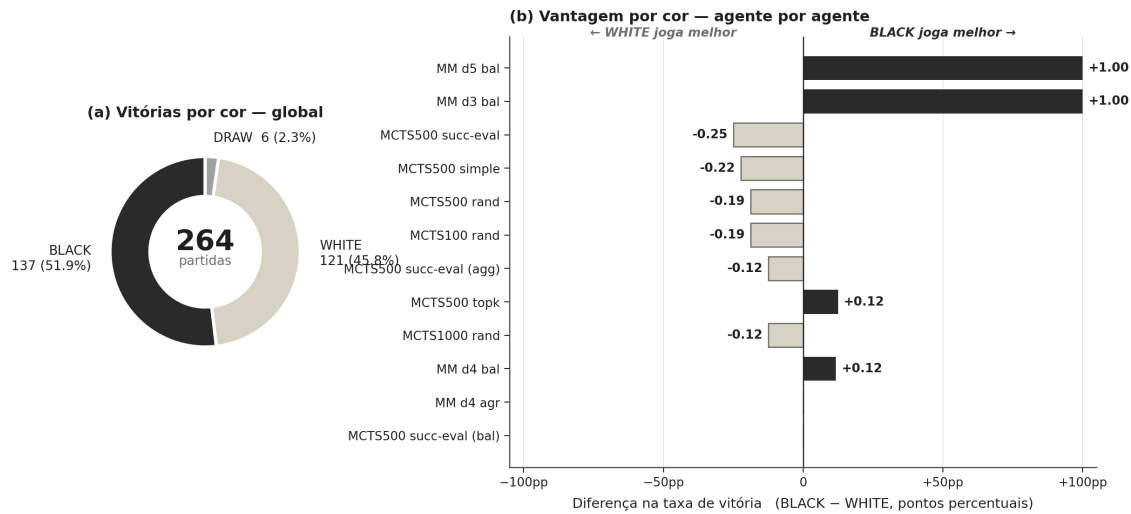


Figura 8. Análise de viés de cor. (a) Distribuição global das vitórias. (b) Diferença BLACK–WHITE na taxa de vitória de cada agente; cada agente disputou exatamente o mesmo número de partidas como BLACK e como WHITE.

(média 11,5 turnos), o resultado é equilíbrio total (50%/50%). No 6×6 (32,2 turnos), o MCTS já vence 62%, e no 8×8 (61,8 turnos) o MCTS domina 75%. O custo, todavia, cresce dramaticamente: o tempo médio por jogada do MCTS vai de 0,25 s no 4×4 a 17,9 s no 8×8 .

Tamanho do tabuleiro — mesmo confronto, comportamento diferente

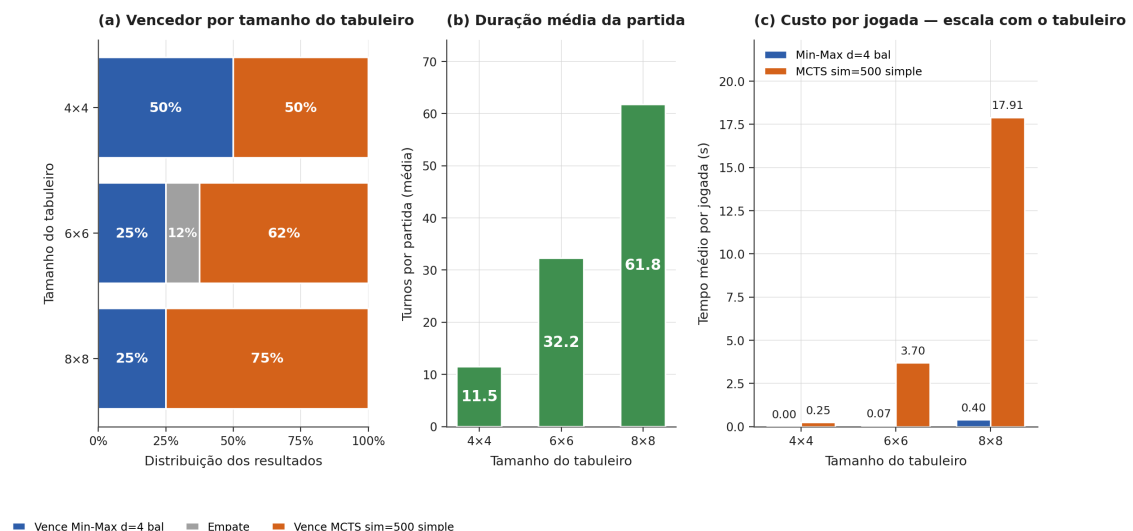


Figura 9. Mesmo confronto em três tamanhos de tabuleiro. (a) Distribuição de resultados. (b) Duração média da partida em turnos. (c) Tempo médio por jogada para cada agente.

A leitura natural é que profundidade fixa do Min-Max captura uma fração cada vez menor da partida conforme o tabuleiro cresce, enquanto o MCTS continua amostrando até

o terminal independentemente do tamanho. Quanto maior o tabuleiro, maior a vantagem relativa do MCTS, ao preço de tempo crescente por jogada.

4.9. Panorama geral das margens

Para encerrar a apresentação dos resultados, a Figura 10 oferece uma visão integrada de todos os principais confrontos do experimento em uma única peça gráfica, com os box-plots ordenados por intensidade da vantagem. A figura permite identificar rapidamente onde há domínio claro (heurística *aggressive* no Min-Max, mais simulações no MCTS), onde há disputa equilibrada (Min-Max balanceado em diferentes profundidades) e onde a relação custo×qualidade favorece um lado em particular.

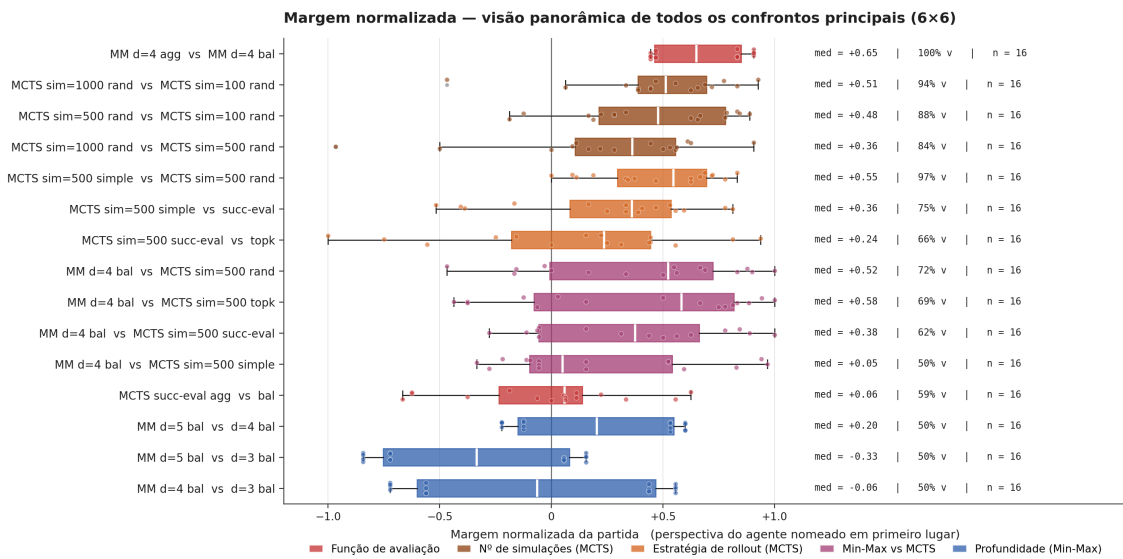


Figura 10. Panorama de todas as margens normalizadas dos confrontos principais (6×6). À direita de cada caixa estão a mediana, a taxa de vitória observada e o número de partidas.

5. Discussão

5.1. Busca por heurística *versus* busca por simulação

Os experimentos articulam um contraste conceitual importante entre os dois algoritmos: Min-Max e MCTS dependem de fontes de informação fundamentalmente diferentes. O Min-Max usa a heurística como sua única fonte de valor para estados não-terminais, e por isso é altamente sensível à qualidade dela — como evidenciado pelos 100% de vitória da *aggressive* sobre a *balanced* (Figura 6). O MCTS, em contraposição, deriva valor de simulações até estados terminais e usa a heurística apenas para guiar essas simulações; a mudança de heurística no rollout altera o desempenho de forma muito mais discreta (56%, com margem mediana de apenas +0,06).

Essa diferença explica também a aparente contradição entre os resultados de profundidade do Min-Max (50% em todos os pares) e os de orçamento do MCTS (monotonicamente crescente): aumentar a profundidade do Min-Max sob a mesma heurística não altera o critério de decisão na fronteira da busca, então as decisões só mudam quando o

aprofundamento extra alcança um estado terminal real ou um padrão tático claro. Aumentar simulações no MCTS, ao contrário, sempre reduz a variância das estimativas $Q(c)$, e por isso o efeito é contínuo.

5.2. Limitações observadas

O Min-Max apresenta a limitação clássica de custo exponencial em profundidade: medimos um crescimento de quase $8\times$ no número de nós expandidos entre $d=3$ e $d=5$, mesmo com a poda Alfa-Beta operando. A poda mostrou-se eficaz (taxa de poda em torno de 0,29 em $d=4$), mas perde força em profundidades altas com a ordenação simples adotada, sugerindo que para $d \geq 5$ seriam necessárias técnicas adicionais como *transposition tables*, *killer moves* ou ordenação dinâmica por iteração.

O MCTS apresenta limitações distintas. Primeiro, o custo cresce linearmente com o número de simulações, e cada simulação cresce com a profundidade média do jogo (efeito visível na Figura 9: tempo por jogada do MCTS vai de 0,25 s no 4×4 a 17,9 s no 8×8). Segundo, a qualidade do MCTS depende fortemente da política de rollout: a comparação entre *simple* e *succ-eval* (Figura 5b) mostra que rollouts mais “inteligentes” não são automaticamente melhores — determinismo excessivo no rollout envia as estatísticas e reduz a diversidade dos playouts.

5.3. Quando cada algoritmo se destaca

Os resultados sugerem padrões claros sobre quando preferir cada abordagem. O Min-Max com Alfa-Beta destaca-se em cenários com (i) horizonte de jogo curto em relação à profundidade alcançável (caso do 4×4 e do 6×6 nas profundidades testadas), (ii) heurística confiável e bem calibrada e (iii) restrições de tempo apertadas, dado que as decisões a profundidade moderada saem em milissegundos.

O MCTS destaca-se em cenários com (i) jogos longos em relação ao horizonte de busca disponível (caso do 8×8), (ii) heurística confiável é difícil de definir e (iii) há orçamento de tempo flexível. A propriedade *anytime* do MCTS é especialmente valiosa em configurações práticas: o algoritmo pode ser interrompido a qualquer momento e produzir uma decisão razoável, e o esforço extra sempre melhora a estimativa.

Essas conclusões são consistentes com a literatura geral sobre busca em jogos [1, 5] e oferecem um diagnóstico concreto para a escolha de algoritmos no Othello em particular.

6. Conclusão

Este trabalho desenvolveu e comparou dois agentes para o Othello em tabuleiro 6×6 : Min-Max com poda Alfa-Beta e Monte Carlo Tree Search. Foram conduzidos 264 confrontos diretos cobrindo 18 cenários experimentais, com variação sistemática de profundidade do Min-Max, orçamento de simulações do MCTS, política de rollout, função de avaliação e tamanho do tabuleiro.

Três achados centrais emergem dos resultados. Primeiro, no 6×6 com a mesma heurística *balanced*, profundidades distintas do Min-Max ($d=3, 4, 5$) não geram vantagem em taxa de vitória, embora o custo computacional cresça por um fator de 8 entre os extremos. Segundo, a função de avaliação molda o Min-Max muito mais fortemente do que

molda o MCTS: trocar de *balanced* para *aggressive* converte 50% em 100% de vitória no Min-Max, mas apenas eleva o MCTS de 50% para 56%. Terceiro, o MCTS escala monotonicamente com o número de simulações, mas a política de rollout pode dominar o efeito do orçamento: *rollout_simple* com 500 simulações empata com o Min-Max $d=4$, enquanto *succ-eval* e *topk*, mais sofisticados e mais caros, vencem menos.

Como extensões obrigatórias, foram implementados o tamanho de tabuleiro configurável e um modo de pontuação ponderada por posição, ambos compatíveis com os dois algoritmos. A análise por tamanho de tabuleiro confirma que a vantagem relativa do MCTS sobre o Min-Max cresce com o horizonte do jogo, em coerência com a teoria.

Trabalhos futuros podem incluir: (i) ordenação de jogadas mais elaborada no Min-Max (*killer moves*, iterativo aprofundamento, tabelas de transposição); (ii) políticas de rollout treinadas a partir de auto-jogo; (iii) comparações com agentes neurais como AlphaZero adaptados ao 6×6 ; e (iv) avaliação no modo *weighted* de pontuação, analisando se o objetivo posicional altera as conclusões sobre profundidade e simulações.

A. Apêndice: Trace do Min-Max com poda Alfa-Beta

A título ilustrativo, apresentamos um trace de execução do Min-Max com poda Alfa-Beta a partir do estado inicial padrão do Othello 6×6 , com limite de profundidade igual a 2, heurística *balanced* e ordenação de jogadas habilitada.

A Figura 11 mostra a árvore de decisão completa construída pelo algoritmo. Nós MAX são representados por triângulos azuis e nós MIN por triângulos invertidos laranjas; folhas avaliadas pela heurística aparecem em caixas verdes com seu valor numérico, e ramos podados aparecem em vermelho tracejado com indicação “–” (não avaliado). O caminho que realiza o valor Min-Max da raiz está destacado em verde.

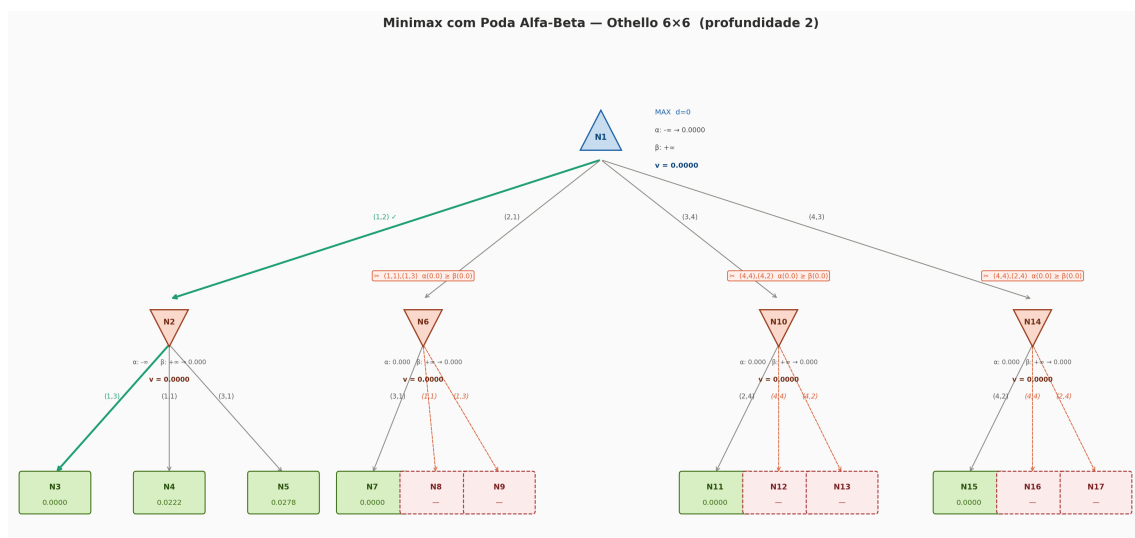


Figura 11. Árvore de decisão completa do Min-Max com poda Alfa-Beta para o estado inicial do Othello 6×6 , profundidade 2, heurística *balanced* com ordenação de jogadas. Foram expandidos 10 nós e ocorreram 3 eventos de poda (em N6, N10 e N14), descartando 6 ramos. A jogada selecionada na raiz é (1, 2) com valor Min-Max 0,0000.

A leitura da árvore funciona como segue. A partir do nó raiz N1 (MAX), são

exploradas em ordem as quatro jogadas legais (1, 2), (2, 1), (3, 4) e (4, 3). A primeira filha N2 é expandida normalmente: todos os seus três sucessores N3, N4 e N5 são avaliados (valores 0,0000, 0,0222 e 0,0278, respectivamente), e o mínimo 0,0000 sobe para N2. Esse valor então define $\alpha = 0,0000$ na raiz. Ao explorar as filhas seguintes N6, N10 e N14, basta encontrar um único sucessor cuja avaliação seja $\leq 0,0000$ para que a condição $\alpha \geq \beta$ seja satisfeita e os demais ramos sejam descartados. É exatamente o que ocorre nos três casos: N7, N11 e N15 retornam 0,0000, e os ramos N8/N9, N12/N13 e N16/N17 sequer chegam a ser explorados, conforme indicado pelas arestas tracejadas vermelhas na Figura 11.

As três podas ocorrem com a condição $\alpha = \beta = 0,0000$, o que se deve à estabilidade do valor heurístico no estado inicial: como peças, mobilidade e cantos estão simétricos no início do jogo, a maioria das avaliações em profundidade 2 retorna valores muito próximos de zero, maximizando a oportunidade de poda.

Referências

- [1] Russell, S.; Norvig, P. (2020). *Artificial Intelligence: A Modern Approach*, 4th Edition. Pearson.
- [2] Knuth, D. E.; Moore, R. W. (1975). “An analysis of alpha-beta pruning”. *Artificial Intelligence*, 6(4):293–326.
- [3] Coulom, R. (2007). “Efficient selectivity and backup operators in Monte-Carlo tree search”. In: *Computers and Games (CG 2006)*, LNCS 4630, pages 72–83.
- [4] Kocsis, L.; Szepesvári, C. (2006). “Bandit-based Monte-Carlo planning”. In: *ECML 2006*, LNCS 4212, pages 282–293.
- [5] Browne, C. B.; Powley, E.; Whitehouse, D.; Lucas, S. M.; Cowling, P. I.; Rohlfshagen, P.; Tavener, S.; Perez, D.; Samothrakis, S.; Colton, S. (2012). “A survey of Monte Carlo tree search methods”. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):1–43.
- [6] Buro, M. (2003). “The evolution of strong Othello programs”. In: *Entertainment Computing: Technologies and Applications*, pages 81–88. Springer.